

PROJECT 2: COURSE PROJECT PROPOSAL

COVER PAGE

Student Name: [Your Name]

Course Title: Web Development II (Back-End Web Development)

Project Title: AURA — A Curated UX/UI Asset Catalog for Web Creators

Submission Date: May 20, 2026

PROPOSAL

1. Introduction

Modern web development relies heavily on reusable, high-quality design assets—such as custom glassmorphic CSS components, SASS mixins, custom HSL color palettes, and SVG icons. **AURA** is a curated, premium digital asset catalog designed to host, showcase, and distribute these assets.

While the front-end will be styled with high-end glassmorphism, responsive grid layouts, and micro-animations, the focus of the application is a robust, secure, and snappy back-end built on Node.js, Express, and MongoDB.

2. Goals and Objectives

- **Database-Driven Catalog:** Store and serve metadata for hundreds of assets, organizing them by category, tags, downloads, and creator.
- **Secure Authentication:** Provide a secure gate for users to create accounts, sign in, manage their profiles, and save their favorite assets.
- **Data Validation & Integrity:** Build a submission portal where verified creators can submit new design assets. The form will feature robust front-end and back-end validation to prevent database clutter.
- **Performance & Offline Capability (PWA):** Implement caching, session persistence, and service workers to make the catalog load instantly and be browsable offline.

3. Target Audience

- **Web Developers:** Looking for copy-paste SASS mixins, responsive layouts, or CSS effects to speed up their projects.
- **UI/UX Designers:** Wanting to share or catalogue custom assets, colors, and patterns.
- **Students & Hobbyists:** Seeking inspiration and free, optimized components for their web applications.

4. Key Features

- **Dynamic Asset Catalog:**
 - Responsive grid interface showing assets.
 - Search and filter capability based on tags (e.g., #glassmorphism, #buttons, #grids, #gradients).
 - Individual asset pages with code copy fields, visual preview modals, and download counters.
- **User Registration & Session Management:**
 - Secure sign-up and login with hashed passwords (using bcrypt).
 - "Favorite" system allowing logged-in users to save assets to their personal dashboard.
 - Express-session and cookies for keeping users logged in across visits.
- **Asset Submission Portal (Forms & Validation):**
 - Form allowing creators to submit new asset codes, titles, tags, and preview image links.
 - Server-side validation using Mongoose schemas and custom middleware to ensure correct code formatting, valid URLs, and anti-spam limits.
- **PWA Features:**
 - Offline mode allowing users to browse previously viewed assets without internet.
 - Add to Home Screen capability on mobile devices.

5. Technology Requirements

- **Front-End UI:** HTML5, SASS (nested components, variables, mixins for glassmorphism effects), Bootstrap 4 (for responsive grid foundations), and Vue.js (for reactive UI state, instant search filtering, and code-copy events).

- **Back-End Runtime:** Node.js and Express.js for the server and routing.
- **Database:** MongoDB (NoSQL) managed via Mongoose ODM for structuring schemas (User, Asset, Category).
- **Authentication:** Passport.js or custom express-session middleware with `bcrypt` for password hashing.
- **Testing:** Mocha/Chai or Jest for unit testing API endpoints (e.g., validating database models and form submissions).
- **Security & SSL:** HTTPS configuration and HTTP-only cookies to protect session IDs.